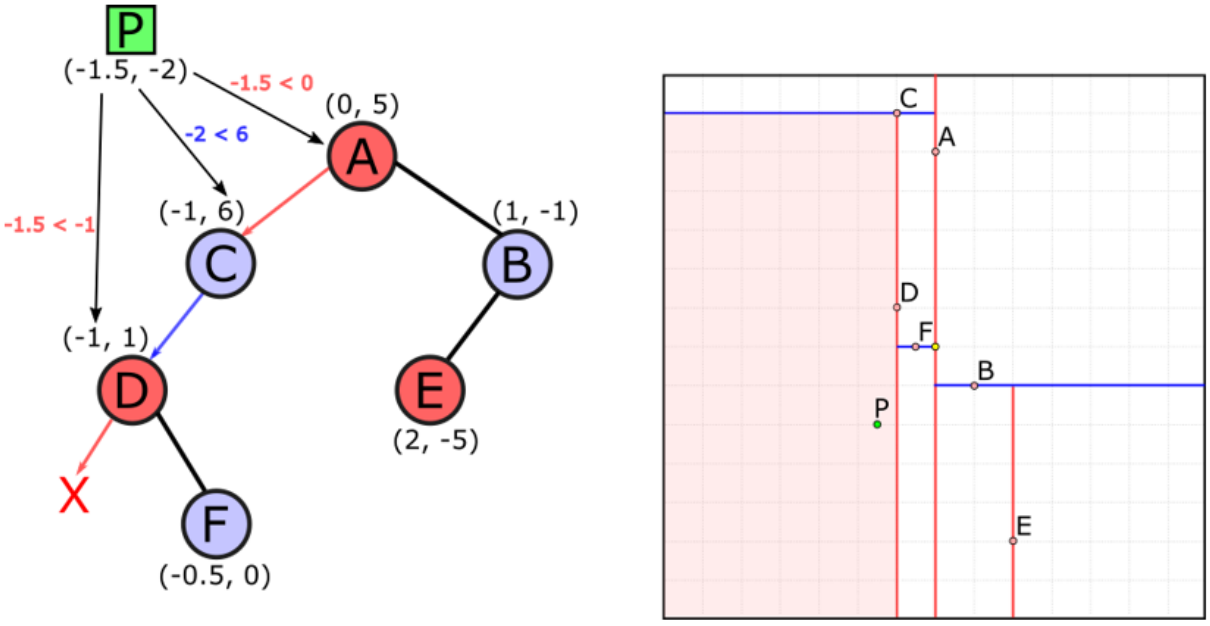


Figure 9.9 An example of an unsuccessful search on a k-d tree (2-D). The searched point, `P`, would ideally lie in the region highlighted in red, which corresponds to the left subtree of node `D`.

Listing 9.3, instead, shows the pseudocode for the search method. The pseudo-code for all these methods will assume that we are providing an internal version that takes a `KdNode` as argument. The public API for `KdTree`, instead, will provide adapter methods that will call the internal ones, for instance `KdTree::search(target)` will call `search(root, target)`.

This way we have more flexibility in reusing these methods (for instance we can run search on just a subtree, not the whole tree).

Notice that this recursive implementation is eligible for tail-recursion optimization, on those languages and compilers supporting it (check appendix E for an explanation of tail-recursion).